

RMIT UNIVERSITY

NTG-agent-vnext

Capstone Final Report

OENG1185 — Capstone Project B

QUOTA team • Industry partner: NashTech

Vu Gia An	s3926888
Nguyen Tien	s3924427
Tran Nguyen Quy Khang	s3929513
Nguyen Dinh Minh Chau	s3923010

Academic supervisor

Tran Nhat Quang

Industrial supervisors

Nguyen Le Quynh Thien — Technical Manager

Nguyen Sieu Anh — Technical Architect

Review draft • 20 September 2026

Evidence snapshot: 20 September 2026.

Contents

1	Executive summary	1
2	Project background and problem statement	2
2.1	Organisational context and the starting point	2
2.2	Why the application is needed	2
2.3	Stakeholders, needs and expected value	3
2.4	Problem statement and project objectives	4
2.5	Constraints and significance	4
3	Literature review and market research	5
3.1	Review scope and analytical approach	5
3.2	Grounding answers in external knowledge	5
3.3	Knowledge boundaries and specialist orchestration	6
3.4	Platform alternatives and implementation fit	7
3.5	Interaction protocols and reusable task skills	8
3.6	Evaluation synthesis and design questions	9
4	Solution design	10
4.1	Scope evolution and design priorities	10
4.2	System architecture	11
4.3	Knowledge ownership and authorised delegation	12
4.4	Provider configuration and capability checks	13
4.5	Skills and interactive responses	14
4.6	Operability and development support	15
4.7	Delivery schedule and team responsibilities	16
5	Result analyses and discussions	20
5.1	Delivered capabilities and engineering findings	20
5.2	Evaluation evidence and trial conditions	21
5.3	Retrieval coverage and consistency	22
5.4	Latency tradeoffs and cache conditions	23
5.5	Comparison with planned success criteria	24
5.6	Priorities for further validation	24
6	Summary and reflection	25
	References	26
	Appendices	27
	Appendix A. Implementation evidence map	27
	Appendix B. Retrieval data and reproducibility	28
	Appendix C. Report production and final review	28
	Appendix D. GitHub history and timeline method	29
	Meeting journals	30

List of Figures

Figure 1	System overview derived from the inspected implementation. Dedicated LightRAG containers use agent-specific workspaces within shared PostgreSQL infrastructure.	11
Figure 2	Authorised delegation: the inner agent performs retrieval within its own knowledge context and returns an answer. The outer agent does not directly query the inner knowledge base.	12
Figure 3	Feature-level Gantt: retrieval, orchestration and interface foundations. Each row identifies contributors, its parent PR and the integration date. Source: GitHub commit and PR records; the selected SHA mapping is retained with the report.	16
Figure 4	Feature-level Gantt: product completion and hardening. Hatched rows show planned phases; coloured rows show selected change-date spans; filled diamonds mark merges into main. Planned dates use the completion plan's prose and milestone table.	17
Figure 5	Reported retrieval coverage by mode and query set. The dashed line is the planned hit@5 target, not a hit@1 requirement. Pilot values were unchanged in the warm-cache rerun. Source: retrieval trials, pp. 2–7.	22
Figure 6	Reported relative median retrieval latency. Each trial has its own naive = 1 baseline; bar differences across trials are not absolute time savings. Source: retrieval trials, pp. 4–7.	23

List of Tables

Table 1	Stakeholder needs derived from project requirements and implemented workflows. Expected value is a design rationale, not a measured usability outcome.	3
Table 2	Knowledge-boundary alternatives and their implications for this application.	6
Table 3	Comparison of documented platform approaches. Repository-fit and remaining-work entries are project-specific interpretations, not claims that an alternative lacks those capabilities.	7
Table 4	Design questions synthesised from the literature, platform comparison and project needs.	9
Table 5	Scope evolution from the planning baseline to the current design.	10
Table 6	Planned responsibilities and milestones compared with GitHub integration evidence. A merge verifies integration, not every acceptance criterion.	18
Table 7	Primary contribution areas and supporting changes corroborated by selected GitHub commits. Dates identify parent-PR integration into main.	19
Table 8	Delivered capabilities mapped to the review's design questions and their evidence.	20
Table 9	Conditions for interpreting the reported retrieval measurements. Source: supplied retrieval trials, pp. 1–9.	21
Table 10	Success criteria retained from the completion plan and assessed against the available evidence.	24
Table 11	Source map for the implementation discussion.	27
Table 12	Transcribed aggregate retrieval results. Each latency row is normalised to its own naive baseline. Source: retrieval trials, pp. 2–7.	28
Table 13	GitHub PR register for the feature timeline and contribution analysis.	29

1 Executive summary

NTG-agent-vnext extends NashTech's existing .NET agent application to support configurable, document-grounded specialist workflows. The QUOTA team's capstone addresses a practical need: users require access to relevant organisational knowledge and task assistance, while administrators must control which agents, documents, tools and model configurations are available. A conversational interface alone does not resolve these requirements. Permissions must remain effective during delegation, provider settings must correspond to usable requests, and structured tasks need an interface that can preserve inputs and progress.

The solution connects a .NET orchestration backend and administration interface with LightRAG knowledge services and an AG-UI conversational client. Each knowledge-enabled agent has a dedicated retrieval context. An outer agent can invoke an authorised inner agent as a tool; the inner agent queries its own knowledge base and returns an answer. Access is checked when exposing the tool and again when invoking it. Administrators can also manage provider model catalogues, test thinking compatibility through the runtime client-construction path, and bind skills containing instructions and validated interface templates.

The reviewed development history substantiates these contributions across twelve GitHub pull requests and twenty-two feature workstreams. Agents-as-tools reached main on 19 June, AG-UI on 7 July, the core knowledge branch on 17 July and role-based access on 23 July. Subsequent integration delivered VM/gateway work on 20 August, skills on 25 August, provider/model management on 3 September and installation automation on 13 September. The record also identifies cross-support between members, including provider routing, legacy-stack removal and macOS setup. These are integration milestones; operational acceptance is assessed separately.

The supplied retrieval trials provide an initial quantitative result. Across twelve pilot questions, all four LightRAG modes achieved a reported hit@5 of 0.917, exceeding the completion plan's numerical threshold of 0.80 on that task set. A warm-cache rerun preserved the reported hit rates. The cold pilot nevertheless showed higher relative median retrieval latency for graph-based modes without higher aggregate hit@5, making the simpler mode a useful baseline for similar questions. A question missed by every mode supplies a concrete diagnostic case. The findings support retrieval coverage and repeated-query consistency under the supplied configuration; they do not establish human-rated answer correctness or performance on a representative enterprise workload.

Delivery was selective rather than uniform across the original ambitions. The implementation retains per-agent knowledge ownership instead of the planned many-to-many shared knowledge spaces, and the earlier long-term memory/search stack was removed. Access control and skills were integrated before the 26 August Phase 2 milestone; the expanded provider workflow merged eight days afterwards. Human answer assessment and several latency, cost and reliability targets remain to be verified. The practical contribution is therefore an integrated and traceable platform extension, supported by bounded retrieval evidence, rather than a new retrieval algorithm or a claim that all success criteria were achieved.

The report first establishes the application needs and synthesises relevant research and platform alternatives into five design questions. It then explains the architecture, interaction and operational choices, traces delivery and member contributions, and analyses the available results. The final reflection identifies lessons from access-aware delegation, provider compatibility, reusable interfaces and developer onboarding, with validation priorities for continued development.

2 Project background and problem statement

2.1 Organisational context and the starting point

NTG-agent-vnext is the QUOTA team's capstone contribution to NashTech's NTG Agent platform. The starting repository already provided a C# conversational application with agent configuration, model-provider connections and supporting services. The team's task was to develop this foundation into a more configurable environment for document-grounded assistants, specialist delegation and interactive workflows. The original application and the capstone extensions therefore have different scopes: the report attributes the latter through the reviewed GitHub history rather than claiming authorship of the entire platform. [1], [2]

The completion plan describes four initial ambitions: a richer document-search pipeline, layered memory, planner-and-worker coordination, and an interface that could compose structured responses. It also records a subsequent concentration on LightRAG, assistants-as-tools, skills and role-based access. [3] These changes show that the project problem was broader than connecting a chat box to a model. It concerned how several assistants could share an application environment while retaining distinct responsibilities, knowledge and permissions.

For a document-grounded assistant, organisational information must be made available at the point of answering. RAG addresses part of this need by retrieving external evidence to condition generation. [4] However, retrieving a relevant document is only one stage of the user journey. The application must also determine which assistant a user can invoke, which documents that assistant searches, whether the selected provider configuration works, and how a multi-step task is presented to the user. NTG-agent-vnext addresses this combination of administration and runtime behaviour.

2.2 Why the application is needed

Consider an illustrative use case derived from the project's requirements: a user asks a general assistant for help with a technical question that requires a specialist's document collection. The general assistant should be able to consult an appropriate inner agent when the user has access. The inner agent should retrieve from its own knowledge base and return an answer. If access is denied, the general assistant must not become an indirect route around the restriction. This scenario explains the requirement for both specialisation and permission-controlled delegation; it is not presented as a recorded sponsor interaction.

A second requirement-derived scenario concerns administration. An administrator adds a model endpoint, fetches the models or deployments available to that endpoint, and enables a model for an agent. A model appearing in a catalogue does not establish that every optional request parameter will be accepted. The administrator therefore needs feedback about the selected configuration before enabling thinking mode. GitHub history makes this need concrete: provider routing was revised in PR 281, while PR 287 replaced a curated thinking-capability list with a live request probe. [2]

A third scenario concerns task interaction. A user may need to provide several fields, compare options and review a structured result. Repeatedly exchanging these values as free text can leave the application without an explicit representation of the current step or selected values. The project's skill templates and streamed UI events supply that representation. The included travel-planning and ticket-booking packages illustrate the interaction pattern with synthetic demonstration data; they are not evidence of real reservations or completed business transactions.

2.3 Stakeholders, needs and expected value

The requirements imply four operational perspectives. End users need a coherent route from a question to relevant assistance. Administrators need control over what is published and available. Skill authors need a manageable way to express task instructions and interaction structure. Developers and maintainers need to reproduce the environment and diagnose integration failures. Table 1 connects these needs to concrete application responsibilities.

Stakeholder	Need or difficulty	Application responsibility	Expected value
End user	Find help from appropriate documents or specialist capabilities without manually navigating every backend.	Published-agent discovery, authenticated conversation, authorised delegation and readable tool results.	A continuous interaction with explicit permission boundaries.
Agent administrator	Configure knowledge, roles, tools, providers and available models consistently.	Agent-scoped knowledge services, access bindings, provider catalogues and capability probes.	Configuration decisions that are visible and actionable.
Skill author / administrator	Add repeatable task guidance and structured input without a bespoke frontend component for each variation.	SKILL.md packages, validated surface templates and per-agent bindings.	Task-specific extension through reviewed configuration.
Developer / maintainer	Prepare local services, handle container routing and diagnose failures across components.	Setup scripts, provisioning status, gateway routing and implementation documentation.	A more reproducible development and handover process.

Table 1: Stakeholder needs derived from project requirements and implemented workflows. Expected value is a design rationale, not a measured usability outcome.

These needs are connected. Publishing an agent before its knowledge service is ready creates a user-facing failure even if the chat interface itself is available. Providing a model selector without usable endpoint configuration moves an integration problem into the conversation. Allowing a skill to render a form whose fields cannot accept input undermines the workflow even when the model produces plausible text. The application therefore requires consistency across its administrative decisions, backend services and client presentation.

The reviewed history contains concrete responses to these difficulties. LightRAG provisioning became a background process with status tracking; gateway routing replaced a host-port ledger; skill validation checked template structure and data bindings; and setup automation prepared dependencies and default-agent configuration. [2] These changes substantiate the practical engineering need for the application. They do not require an invented claim about how many staff hours were lost or how many employees adopted the system.

The distinction between platform value and measured benefit remains important. The project supplies mechanisms that can reduce repeated configuration and interface work, but no controlled productivity study or representative user evaluation has been supplied. The appropriate claim is that the platform implements the capabilities needed to support those workflows. The extent of improvement in staff efficiency, task success or administrator workload is a subsequent empirical question.

2.4 Problem statement and project objectives

The project addresses the following problem: **how can an existing .NET agent application support configurable, document-grounded specialist workflows while preserving caller permissions and maintaining consistent behaviour across model providers and user interfaces?** The difficulty lies in connecting these requirements. A knowledge boundary must survive delegation; a provider option must correspond to a usable request; and a task instruction must reach an interface capable of rendering its intended interaction.

Five objectives make the problem assessable. First, each knowledge-enabled agent should have a defined retrieval context and a document lifecycle that distinguishes upload from indexing readiness. Second, the runtime should expose and invoke specialist agents according to the caller's permissions. Third, administrators should manage available models and test optional thinking behaviour through the runtime request path. Fourth, skills and structured UI events should support reusable task flows. Fifth, the project should leave a reproducible setup process and evidence with which these capabilities can be evaluated.

These objectives also define boundaries. The project does not develop a new foundation model or retrieval algorithm. It adopts existing frameworks and protocols, then contributes the integration, configuration and lifecycle logic needed in this application. The retained design does not implement the completion plan's proposed many-to-many shared knowledge spaces, and removal of the earlier long-term memory backend is a documented scope decision. Likewise, demonstration skills are not integrations with live booking or enterprise transaction systems. [2], [3]

2.5 Constraints and significance

The work operates within an existing .NET application, a capstone delivery schedule and externally provided model services. Per-agent knowledge services introduce container and storage coordination, while the conversational frontend adds another process and protocol boundary. NashTech's infrastructure support and the university's Claude API allocation helped the team develop and test within these constraints. The reported setup friction also made local reproducibility a practical requirement rather than an incidental documentation task.

The project's significance is its treatment of the assistant as an administered application. The administrator defines the available capabilities; the runtime applies those choices during delegation and retrieval; and the client renders supported interactions. This is useful beyond a single demonstration because future assistants can reuse the same configuration and integration mechanisms while differing in their documents, permitted users and task guidance. The claim is a reusable engineering approach, not established superiority over every alternative platform.

The completion plan provides measurable targets for retrieval, answer quality, ingestion, response latency, availability and cost. [3] GitHub integration records establish when selected capabilities entered main, while the supplied retrieval trials establish a narrower set of observed outcomes. The report uses both forms of evidence: implementation and contribution records explain what was delivered; experimental results examine how the tested retrieval configuration behaved. Section 3 derives the design questions from prior work, Section 4 explains the solution and its development, and Section 5 assesses the available results against the retained objectives.

3 Literature review and market research

3.1 Review scope and analytical approach

The review examines how existing methods and platforms address five concerns arising from the project: access to external knowledge, specialist orchestration, permission boundaries, configurable interaction, and evaluation. Research papers establish the methodological basis, while official framework and protocol documentation establishes the capabilities of reusable components. The comparison focuses on fit with the project's requirements; it is not an exhaustive survey or a measured ranking of products.

The distinction between these sources matters. A research result obtained on a published task set does not establish the performance of the team's corpus or configuration. Similarly, documentation of a supported capability does not prove that the NTG integration exercises it correctly. The review therefore moves from prior work to a design implication and then to an evaluation question that the project can address.

3.2 Grounding answers in external knowledge

Lewis et al. combine a parametric generator with retrieval from a non-parametric knowledge source. Their work establishes a basis for generating answers using evidence obtained at inference time. [4] For NTG-agent-vnext, the implication is to represent retrieval as an explicit service boundary. Documents can then be managed independently of model training, and the evaluation can inspect both the retrieved material and the answer that uses it.

A straightforward alternative is to place a larger amount of source material directly into the model's context. This can simplify small demonstrations, but context capacity alone is not evidence of effective evidence use. Liu et al. found that performance on their long-context tasks varied with the position of relevant information, with weaker use of information in the middle of the context. [5] This finding motivates testing evidence selection and use separately. It does not establish that the same positional effect occurs in the project's GPT-5.1 configuration, which was not evaluated by that study.

Graph-oriented approaches address a different aspect of retrieval. Edge et al.'s GraphRAG constructs entity graphs and community summaries to support global questions across a corpus. Its reported improvements concern global sensemaking and summary coverage under the study's conditions. [6] LightRAG also uses graph structure, combining entity and relationship information with vector retrieval and a dual-level retrieval design. [7] These methods make relational and broader contextual questions relevant comparison cases; they do not imply that graph processing improves every direct lookup.

For this project, the central distinction is between retrieving a specific passage and constructing an answer that draws on relationships across information. The former may be served efficiently by a simpler retrieval mode. The latter motivates testing graph-based alternatives. A useful corpus should therefore contain both kinds of questions, with an explicit account of which task types are represented. The supplied mixed twelve-query pilot provides an initial comparison, but it cannot establish the preferred mode for all intended organisational work.

The design implication is to retain a simple baseline and compare it with more elaborate modes under matched conditions. Additional indexing and retrieval stages should be justified by their contribution to the task, including response quality and processing cost. The team's use of LightRAG's built-in modes is consequently assessed as a bounded integration choice; the original plan's additional keyword and reranking stages are not treated as necessary merely because they were initially proposed.

3.3 Knowledge boundaries and specialist orchestration

Retrieval relevance and retrieval permission answer different questions. A result may be highly relevant yet outside the information scope intended for a particular assistant or caller. Ferraiolo and Kuhn’s role-based access-control model provides a foundation for organising permission around organisational roles. [8] Applied to NTG-agent-vnext, the relevant design question is where the access decision is enforced when an assistant invokes another capability. Role configuration alone would be insufficient if indirect invocation bypassed it.

The project considered document-tag separation and shared knowledge spaces before retaining per-agent LightRAG contexts in the inspected implementation. The following comparison is an engineering analysis of those alternatives, informed by the project requirements and source paths. It does not claim that a single tenancy model is universally more secure. [1], [3]

Boundary strategy	Potential benefit	Responsibility or tradeoff
Shared index with filtering	Shared indexing and storage can support reuse across assistants.	Every relevant query path must apply the intended scope correctly; tags are useful only when the retrieval layer enforces them.
Per-agent retrieval context	Knowledge ownership and routing align with the configured agent.	Container lifecycle, workspace configuration and shared infrastructure remain operational responsibilities.
Many-to-many knowledge spaces	Several agents can reuse curated collections without copying every collection.	Access must account for both agent permissions and collection bindings; this planned topology is not established by the current implementation.

Table 2: Knowledge-boundary alternatives and their implications for this application.

ReAct demonstrates the interleaving of model reasoning and actions that obtain information from an external environment. [9] The relevant connection is explicit capability invocation: a specialist agent can be exposed through a tool interface while performing its own task. NTG-agent-vnext uses this pattern to give the outer agent a callable specialist, rather than direct access to the specialist’s retrieval client.

This choice can be compared with a single agent holding all tools, or with an explicit planner-and-worker workflow. A single agent reduces delegation boundaries but concentrates tool choice and knowledge responsibilities. A workflow can make steps and state transitions explicit, at the cost of defining and maintaining the coordination logic. Tool-based delegation fits the project’s retained scope because each specialist can answer a subtask behind a common invocation contract. This rationale follows from the final design; no comparative orchestration-performance experiment was supplied.

The contract nevertheless has costs. A delegated call can add model and retrieval work, and the outer agent must interpret the child’s response. Returning an answer also compresses the underlying evidence, making provenance and error handling important. These tradeoffs lead to two questions for the evaluation: does the runtime enforce caller access at both tool exposure and invocation, and does the composed answer preserve the relevant evidence from the specialist? The inspected code addresses the first structurally; the second requires judged end-to-end tasks.

3.4 Platform alternatives and implementation fit

The platform decision concerns what to adopt as a foundation and what remains application-specific. Dify provides an application-building environment with workflows, RAG facilities and model/tool integration. Microsoft Agent Framework supplies agent and workflow abstractions for application development, including the .NET environment used by the project. LangGraph emphasises stateful orchestration and durable execution. [10], [11], [12] These options occupy different layers, so their documented strengths are compared along the same project-relevant dimensions in Table 3.

Dimension	Dify	Microsoft Agent Framework	LangGraph
Adoption unit	An application platform and associated workflow environment.	Agent and workflow building blocks integrated into application code.	A stateful orchestration framework integrated into application code.
Workflow emphasis	Configured application workflows and knowledge/model capabilities.	Agent construction, tools and workflow integration.	Explicit state, durable execution and workflow control.
Fit with this repository	Adoption would require mapping the existing administration and agent model to the platform.	Matches the existing .NET runtime and allows the project's own configuration model to remain central.	A possible orchestration alternative; its state model would require additional application integration.
Work still required for NTG	Confirm how agent-level knowledge and role requirements map to the adopted configuration.	Implement role checks, scoped knowledge binding, provider administration and client contracts.	Implement equivalent access, knowledge, provider and client integrations around the workflow.

Table 3: Comparison of documented platform approaches. Repository-fit and remaining-work entries are project-specific interpretations, not claims that an alternative lacks those capabilities.

Continuing with Microsoft Agent Framework can therefore be justified by continuity with the repository and control over the existing administrative model. It should not be justified by an untested claim that the framework is faster or more capable than the alternatives. The reusable framework supplies agent execution primitives; the capstone contribution supplies application decisions such as which specialist is visible, which knowledge context it receives and how an administrator configures its model.

Provider flexibility introduces another integration boundary. The GitHub record documents endpoint-routing fixes, global provider configuration, curated model selection and finally live thinking probes. [2] These changes support a practical distinction between catalogue discovery and request compatibility. The former identifies what an endpoint advertises; the latter checks a selected option through an actual request path. A successful probe still does not compare the quality of different providers' reasoning, and its validity is bounded by the tested endpoint and configuration.

The market-review conclusion is consequently an integration gap specific to this project, rather than a claim that configurable assistants are absent from the market. Existing platforms and frameworks supply substantial capabilities. NTG-agent-vnext combines a particular set of requirements within an inherited application: agent-owned knowledge, caller-aware delegation, configurable providers, reusable skills and an interactive client. The report evaluates that combination and the work required to make its boundaries consistent.

3.5 Interaction protocols and reusable task skills

A tool-capable assistant needs a way to communicate more than final text. AG-UI defines an event-oriented connection between an agent backend and a user-facing application, including streamed activity and state interaction. A2UI supplies a declarative representation of interfaces rendered by compatible clients. [13], [14] Their roles are complementary: one concerns the interaction stream, while the other concerns the structure of a surface carried through the application.

MCP addresses a further boundary by exposing tools, resources and prompts through a client–server architecture. [15] A backend tool connection, a browser-facing event stream and a rendered form are therefore different integration contracts. Treating all three as a generic “agent protocol” would conceal where validation and error handling belong. In NTG-agent-vnext, access to an inner agent remains an application-level decision even when other tools use MCP.

The evolution of the project’s interface illustrates the distinction. PR 264 introduced the AG-UI conversational path and generative surfaces. PR 286 then added skill-bound templates and client-capability handling. The earlier per-tool approach required a corresponding frontend component for a new visual result. A declarative catalogue broadened what could be rendered, while authored skill templates constrained the layout for repeatable workflows. [2] These are successive engineering choices, not evidence that every task should use generated UI.

The Agent Skills specification organises reusable expertise around instructions and supporting resources. [16] The project adopts that concept in administrator-managed packages, then adds its own storage, binding and template-validation logic. Its implementation deliberately supports a constrained package format; adopting the skill concept does not imply support for every optional resource or execution capability in other skill ecosystems.

Three interaction strategies emerge. Plain text remains suitable when an answer requires explanation but little structured input. Freely generated surfaces offer flexibility when presentation varies, but require robust handling of invalid or inconsistent structures. Authored templates are appropriate when the workflow and input fields should remain stable while the data change. NTG-agent-vnext retains both declarative paths, using skill templates to reduce repeated layout generation in the supplied demonstrations.

Template validation and task correctness must still be separated. A surface can satisfy a component schema while presenting an incorrect recommendation, and a plausible recommendation can appear in a form with unusable bindings. The project’s validator addresses structural constraints such as references and seeded input paths. Human task assessment and runtime interaction checks address the separate questions of whether the content is correct and the workflow is useful. This division gives a concrete evaluation purpose to the separation between model output, transport and rendering.

3.6 Evaluation synthesis and design questions

RAG evaluation must distinguish whether relevant material was retrieved, whether the answer is supported by that material, and whether it answers the question correctly. RAGAs presents a framework for evaluating several dimensions of RAG pipelines, including context relevance, faithfulness and answer relevance. [17] These dimensions inform the report’s analysis, but RAGAs was not executed for the supplied trials. Its relevance is methodological: one favourable metric should not stand in for the entire application outcome.

The retrieved-source unit also matters. A hit on a document identifier and a hit on an answer-bearing chunk are different observations. Likewise, a correct specialist answer does not establish that the outer agent uses it correctly, and a successful health sample does not measure availability throughout an agreed window. The project’s evaluation should preserve these distinctions in its scorer, task definitions and runtime traces.

Table 4 consolidates the review into questions that guide the design and analysis. Each question links a user need to a chosen mechanism and a corresponding test. This provides a traceable contribution argument: prior work supplies methods and infrastructure, the team supplies application integration, and the evaluation determines which resulting claims are supported.

ID	Design question	Project response	Evaluation implication
DQ1	How can answers use the appropriate agent’s documents?	Agent-scoped LightRAG and a document indexing lifecycle.	Measure retrieved evidence and judged answers separately; verify document ownership and routing.
DQ2	How can specialists be reused without bypassing caller permissions?	Agents-as-tools with access filtering and invocation-time checks.	Exercise allowed, denied, revoked and unpublished-agent cases through direct and delegated routes.
DQ3	How can administrators configure heterogeneous models reliably?	Provider/model catalogues, explicit selection and live thinking probes.	Test configuration and error paths; distinguish accepted parameters from output quality.
DQ4	How can task interaction be extended without per-task frontend redevelopment?	AG-UI events, A2UI rendering and skill-bound authored templates.	Check import validation, usable inputs, state continuity and task correctness.
DQ5	How can the integrated platform be reproduced and assessed?	Setup automation, provisioning status, documentation and retained evidence.	Repeat setup and recovery exercises; retain absolute timings, cost records and evaluation artefacts.

Table 4: Design questions synthesised from the literature, platform comparison and project needs.

The available retrieval results primarily address DQ1’s coverage component and expose latency tradeoffs. The inspected implementation and GitHub history provide evidence for mechanisms relevant to DQ2–DQ5. They do not replace behavioural acceptance tests. The resulting research direction is focused: evaluate representative authorised workflows, explain unsuccessful cases, and measure operational properties under documented conditions before drawing broader deployment conclusions.

4 Solution design

4.1 Scope evolution and design priorities

The completion plan remains the baseline for comparing commitments with the current solution. [3] Scope evolved toward explicit agent-level knowledge ownership and tool-based delegation. Table 5 records the main changes without treating the presence of code as evidence of a particular delivery date. The team confirmed the responsibility areas and the rationale for the isolation, provider and skill changes; the source snapshot supplies implementation evidence. [1]

Area	Baseline or earlier approach	Current design and consequence
Knowledge access	Shared knowledge spaces and document-tag-based separation were considered.	Dedicated LightRAG container/workspace per agent. Authorised access to another agent's knowledge occurs through that agent.
Orchestration	Planner/worker handoff and broader coordination scope.	Inner agents exposed as tools to an outer agent; access checks govern exposure and invocation.
Retrieval pipeline	Additional hybrid keyword/reranking work.	Use of LightRAG retrieval modes; no reranker in the supplied trial configuration.
Memory	Broader layered memory ambitions.	Scope reduced toward recent conversation and summaries; long-term user memory is outside the retained commitment.
User interface	Native Blazor-oriented interface direction.	Separate conversational client, with AG-UI streaming and declarative UI integration.
Configuration	Provider and agent configuration as a platform requirement.	Model discovery, thinking compatibility probes and administrator-defined skills extend the configuration workflow.

Table 5: Scope evolution from the planning baseline to the current design.

The completion plan already described per-agent isolation as implemented, then scheduled a further move to shared knowledge spaces. The current repository retains per-agent contexts; it does not establish delivery of that additional many-to-many topology. Thus the earlier isolation redesign and the later unfulfilled shared-space extension are separate scope events.

The isolation change makes the agent the unit of knowledge configuration and delegated access. This is easier to reason about than treating a document tag as the entire access boundary, but it creates operational responsibilities for container lifecycle, configuration and shared infrastructure. The tradeoff is therefore not isolation without cost: stronger separation in the application topology is accompanied by more resources to provision and monitor.

The original evaluation targets are retained for comparison rather than revised after observing the trials. They cover hit@5 and human-rated quality, ingestion percentiles, time to first token, complete response latency, availability, recovery and cost. Section 5 reports which of these have corresponding measurements. A changed implementation scope does not silently redefine a target as achieved.

4.2 System architecture

The architecture separates administration, conversational interaction, orchestration and knowledge services. Figure 1 presents these responsibilities and their principal relationships. The administrative interface configures providers, agents, access roles, tool bindings and skills. The user-facing client sends requests and receives streamed responses and interface events. The backend constructs the configured agent and mediates its access to model providers and tools.

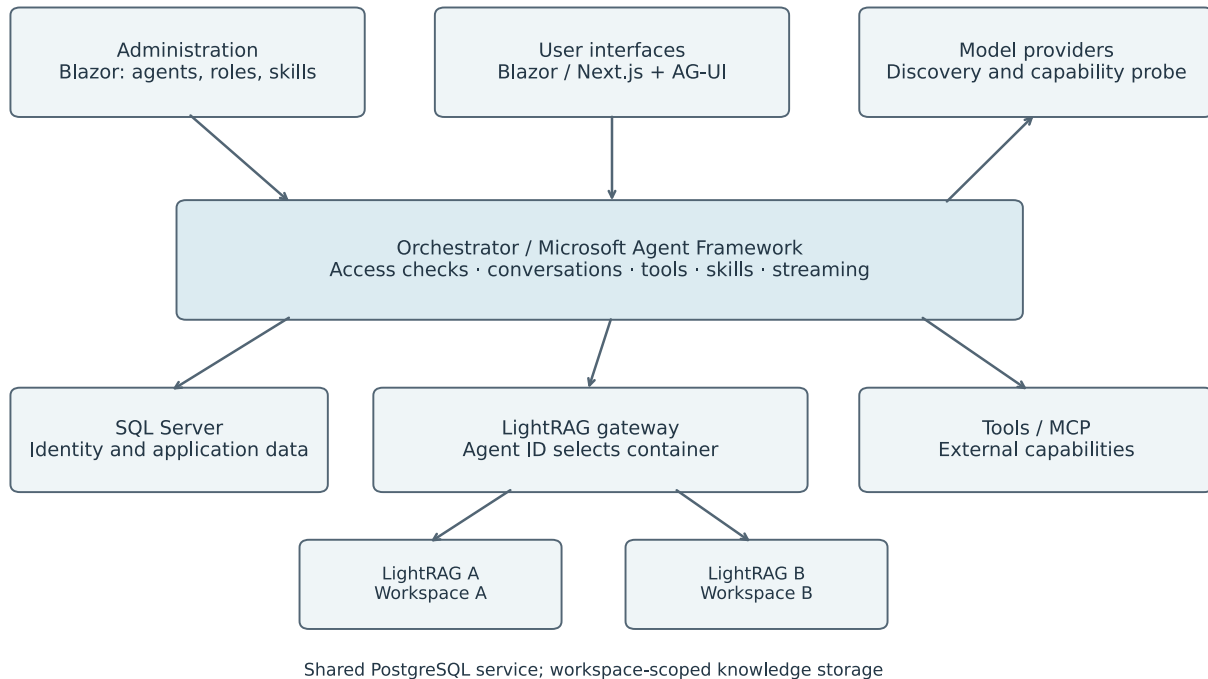


Figure 1: System overview derived from the inspected implementation. Dedicated LightRAG containers use agent-specific workspaces within shared PostgreSQL infrastructure.

Application data and agent configuration are persisted separately from the runtime conversation flow. Knowledge requests pass through an agent-scoped gateway to the relevant LightRAG service. Dedicated containers do not imply physically independent database servers: the inspected configuration uses shared PostgreSQL storage with workspace scoping. This distinction is important when describing the security boundary and the operational consequences of the design.

The platform also integrates external tools, including MCP-connected capabilities. These are reusable building blocks; the team's contribution lies in configuring and connecting them to the agent runtime and administrative workflow. Framework capabilities are therefore distinguished from application-specific work such as determining which inner agents a user can invoke.

Failures can occur at several boundaries: the provider may reject a request, an inner agent may be unavailable, or a knowledge container may need to start before serving a query. The inspected implementation contains access checks, provider error handling and knowledge-service readiness logic. These mechanisms describe intended runtime behaviour. Their existence is not treated as proof of a recovery-time target or complete fault tolerance.

4.3 Knowledge ownership and authorised delegation

Each knowledge-enabled agent receives a LightRAG service addressed through its agent identifier. The container manager derives a workspace identifier for the agent and configures PostgreSQL-backed storage. Document ingestion associates the uploaded original with the agent and document identifiers, then submits processing work to LightRAG and tracks its asynchronous progress. This allows upload acceptance to be distinguished from completed indexing.

Delegation preserves the same ownership model. An outer agent exposes only eligible, published inner agents for the current user. When the delegated tool is invoked, the access service is consulted again, and the child agent receives a knowledge plugin scoped to its own identifier. The inner agent queries that knowledge and returns its answer to the outer agent. Figure 2 illustrates the sequence. [1]

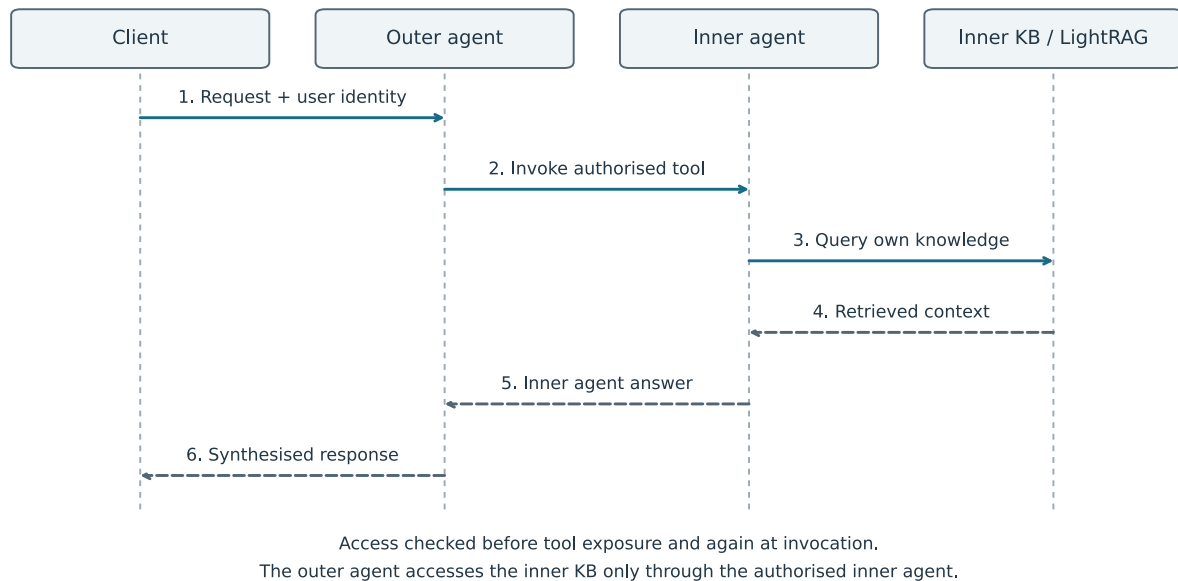


Figure 2: Authorised delegation: the inner agent performs retrieval within its own knowledge context and returns an answer. The outer agent does not directly query the inner knowledge base.

This design separates permission to invoke a specialist from direct access to the specialist’s retrieval client. The access service considers publication state and the configured owner, administrator or role-based permissions. Checking access at tool construction limits exposed capabilities; checking again at invocation reduces dependence on an earlier decision remaining valid.

The remaining validation requirement is behavioural. A convincing access-control evaluation should exercise users with and without the required role, revoked access, unpublished inner agents and cross-agent document requests. It should also test whether returned content follows the intended information-sharing policy. An authorised inner-agent invocation is designed to return information; isolation is therefore a controlled delegation boundary, not a claim that information can never flow between agents.

Container separation and workspace scoping also depend on correct routing and credentials. The report does not claim independent physical tenancy, a penetration-test result or resistance to prompt injection. These are separate assurance questions requiring evidence beyond the inspected control paths.

4.4 Provider configuration and capability checks

Provider configuration separates an endpoint and its available models from the settings of an individual agent. An administrator manages the provider, fetches the catalogue or deployments exposed by it, enables suitable models, and selects one explicitly for each agent. PR 287 introduced the global provider/model management workflow, building on the earlier discovery and routing changes. This separation avoids treating every agent as an independent copy of the provider configuration while retaining agent-specific model choice. [2]

The workflow also separates availability from compatibility. A catalogue response indicates that a model or deployment is advertised through the queried endpoint. Thinking support is tested separately by sending a small request through the same thinking-client construction path used during chat. The implementation uses a 30-second timeout and returns a structured result with an error when the request fails. A successful probe establishes acceptance of the tested request configuration at that time; it does not measure reasoning quality or establish that every optional setting is supported. [1]

The final design emerged through two distinct revisions. PR 281 corrected provider-specific endpoint and reasoning routing and extracted client-construction responsibilities. PR 287 subsequently moved thinking-mode gating away from a curated capability list. Its 26 August commit introduced the live probe, so the final runtime contract is more precise than the PR's earlier summary. This is why the report follows the commit and final code when describing the feature.

Errors are part of the administrative workflow. An invalid endpoint, a discovery failure and a rejected thinking request should lead to actionable configuration feedback rather than an unexplained failure during the user's first conversation. At the same time, a timeout cannot reliably distinguish an unsupported option from an unavailable service. The result therefore includes the error context, and runtime calls still require their own failure handling.

There is also a consistency requirement between probing and use. A probe implemented through a separate request builder could accept a payload that the agent never sends. Reusing the thinking-client construction path reduces that mismatch. The probe is still a small compatibility request with its own input and options; the report does not describe it as a complete replay of every later conversation request.

The implemented workflow answers DQ3 by making provider/model decisions explicit and testable. Its next acceptance checks should cover successful discovery, unavailable catalogues, permission failures, accepted and rejected thinking requests, and saved agent settings. Output quality remains a separate evaluation dimension. This distinction allows administrators to select a usable configuration without representing compatibility as a ranking of model intelligence.

4.5 Skills and interactive responses

A skill is stored as a package of task instructions and supporting assets that an administrator imports and binds to an agent. PR 286 added the storage, administrative API, interface and runtime path for these packages. At the start of a run, the agent receives a compact catalogue of bound skill names and descriptions. The `load_skill` tool supplies the detailed instructions when a skill is needed, while `render_skill_surface` renders an authored template with task-specific values. [1], [2]

This progressive loading has a specific purpose: every bound skill need not contribute its entire instruction body to every request. The runtime guidance also tells the model to reload a skill when continuing its task on a later turn, because the loaded body is not retained as ordinary conversation history. The project therefore treats skill activation and conversation persistence as separate mechanisms, rather than assuming that a previously loaded instruction remains available indefinitely.

The presentation path retains two forms of declarative output. The general `render_a2ui` tool can construct a surface against the supported catalogue. The skill path uses a template authored in advance and changes its data at runtime. For a repeated multi-step flow, the latter keeps the input fields and layout stable while allowing the model to supply the relevant content. The travel-planning and ticket-booking packages demonstrate this pattern using synthetic data.

Validation occurs at import as well as during rendering. The package importer checks archive entries and rejects problematic paths and duplicates. The surface validator checks component structure, references and data bindings. In particular, input paths need seeded values: an apparently valid control can otherwise render without a usable update target. This is a concrete example of an interface failure that cannot be detected by judging the model's text alone.

PR 286 also addresses continuity and client differences. Stable surface identifiers allow an updated skill surface to remain in the existing card, and skill surfaces have a replay path after a page reload. Generative output is gated by what the client can render, avoiding the display of protocol payloads as ordinary assistant text. These changes connect DQ4 to an explicit interaction contract: a surface must remain addressable, its fields must function, and unsupported clients must receive appropriate handling.

The package format is intentionally constrained. Executable scripts are outside the supported skill path, and structural checks do not establish that every instruction is trustworthy. The remaining questions concern content and use: whether a skill chooses the right next step, preserves user selections and produces a correct result. Tests of the importer and surface schema provide one layer of evidence; live multi-turn tasks and human assessment provide the complementary layer. No general prompt-injection resistance or real booking functionality is claimed.

4.6 Operability and development support

The platform's resource model makes lifecycle management part of the user experience. A knowledge-enabled agent may need a container to be provisioned, an indexing job to finish or a service to become ready before a query can succeed. The GitHub record distinguishes these concerns: PR 265 includes per-agent containers, document-status handling and idle shutdown, while PR 282 adds provisioning-state work, VM migration and gateway routing. [2]

The gateway replaces the earlier host-port ledger with agent-addressed routing. This gives the application a consistent route for selecting the intended LightRAG service, while container startup and readiness remain separate responsibilities. Shared PostgreSQL infrastructure stores agent-scoped knowledge through distinct workspace configuration. The resulting topology supports per-agent ownership but still depends on shared services, correct routing and working credentials.

Local installation was developed as an end-to-end workflow. PR 285 adds a bootstrap and setup script that checks prerequisites, obtains the required Azure endpoint and key, generates other local secrets, writes configuration, starts the local knowledge stack and launches the application host. Ubuntu/Debian and macOS have different prerequisite paths; macOS still requires an available Docker environment. Existing configuration is preserved where the script supports reuse. The design reduces the number of manual transitions between setup stages, although no timed onboarding comparison was supplied.

Default-agent provisioning connects setup to the model configuration. The seeder fills an unconfigured default agent from the available LightRAG model settings, enabling a first conversation without a separate provider-entry step. The final implementation checks whether a provider key is already present before filling it, so a configured provider is not indiscriminately overwritten. A September integration change adapted this logic to the provider entity introduced by PR 287. [1], [2]

The workflow history also records supporting maintenance. PRs 278 and 280 add secret-scanning configuration and repair the scanner runtime used by CI. PR 288 updates dependencies and integration code. These changes support continued development, but a configured workflow is not evidence that every merge passed its quality gate. Author-reported checks in PRs are distinguished from independently inspected run records.

Operational evaluation remains a separate deliverable. Telemetry and development health endpoints provide implementation hooks; the retrieval trial supplies sampled health observations. Neither establishes a complete demonstration-window availability record, a backup/restore result or a recovery-time distribution. DQ5 therefore calls for a repeatable fresh-environment setup, recovery exercises and retained runtime measurements. The current contribution is the integrated setup and lifecycle tooling with traceable development history, ready to support those checks.

4.7 Delivery schedule and team responsibilities

4.7.1 Reconstructing the feature timeline

The delivery timeline is reconstructed from twelve GitHub pull requests (PRs), their commit lists and the corresponding integration commits in the local repository. The records cover all four student contributors and selected industrial-supervisor maintenance. The feature mapping identifies twenty-two workstreams rather than treating each large PR as a single feature. Dates are converted to Vietnam time (UTC+7). [2]

Figure 3 shows the foundations of the final solution. A coloured span connects the first and last selected commit-author dates for a feature; points identify the recorded change dates. The dotted segment leads to the PR's merge into main. These intervals describe the available development record, not continuous labour, exact task start dates or deployment acceptance. Shared commits inherited by stacked PRs are excluded from unrelated feature rows.

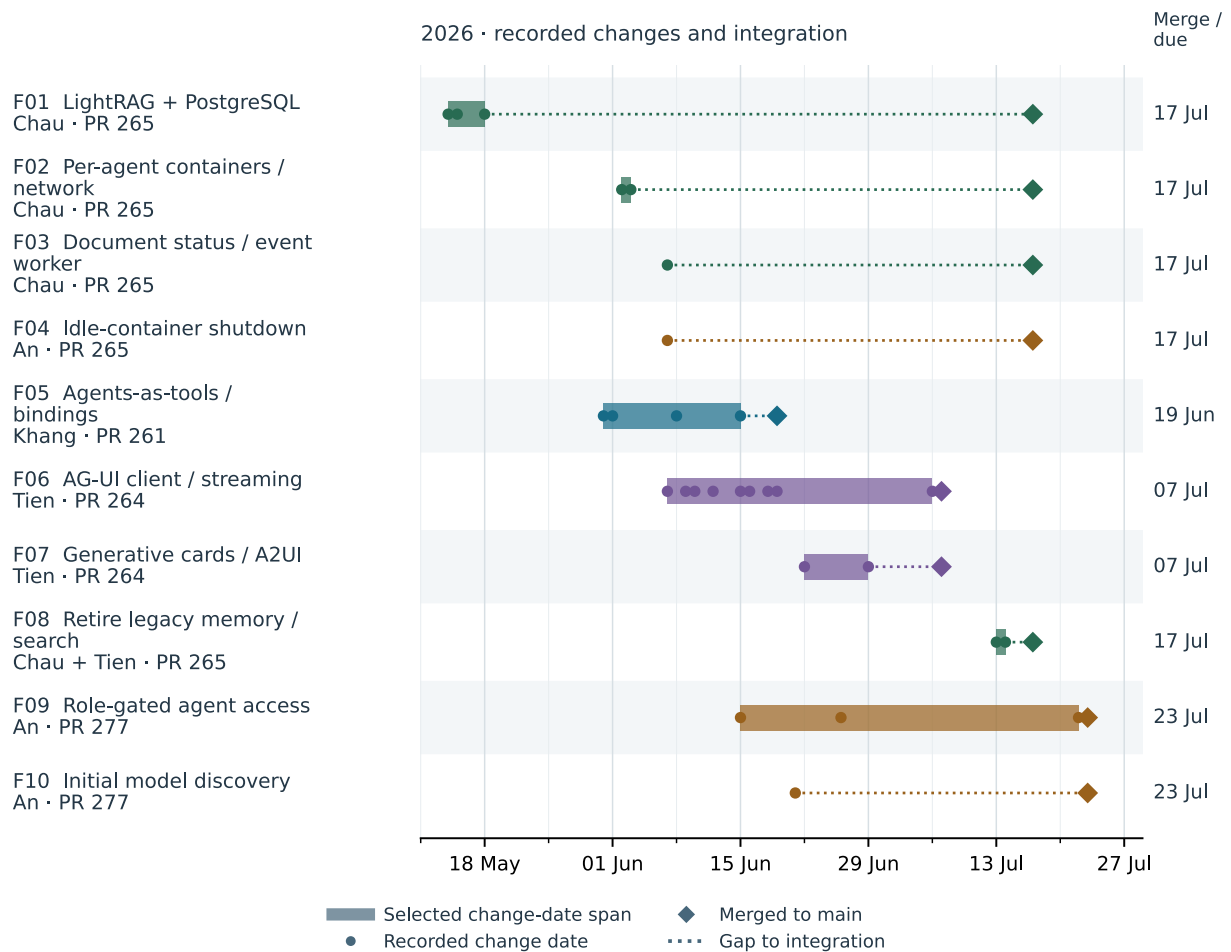


Figure 3: Feature-level Gantt: retrieval, orchestration and interface foundations. Each row identifies contributors, its parent PR and the integration date. Source: GitHub commit and PR records; the selected SHA mapping is retained with the report.

The sequence clarifies how the architecture emerged. Chau's LightRAG and PostgreSQL changes preceded per-agent containers and document-status handling; An added idle-container shutdown. Khang's agents-as-tools work reached main on 19 June, followed by Tien's AG-UI and generative-interface integration on 7 July. The combined knowledge branch merged on 17 July after additional cleanup, including the removal of the legacy memory and search services. Role-based access and an initial model-discovery implementation followed on 23 July. These are integration milestones, not claims that every associated requirement was fully evaluated.

4.7.2 Product completion and integration

Figure 4 places subsequent work against the completion plan's phase windows. It separates provisioning, VM migration and gateway routing, and distinguishes provider catalogue management from the later live thinking probe. This exposes dependencies and supporting work that a phase-only chart would conceal. [2], [3]

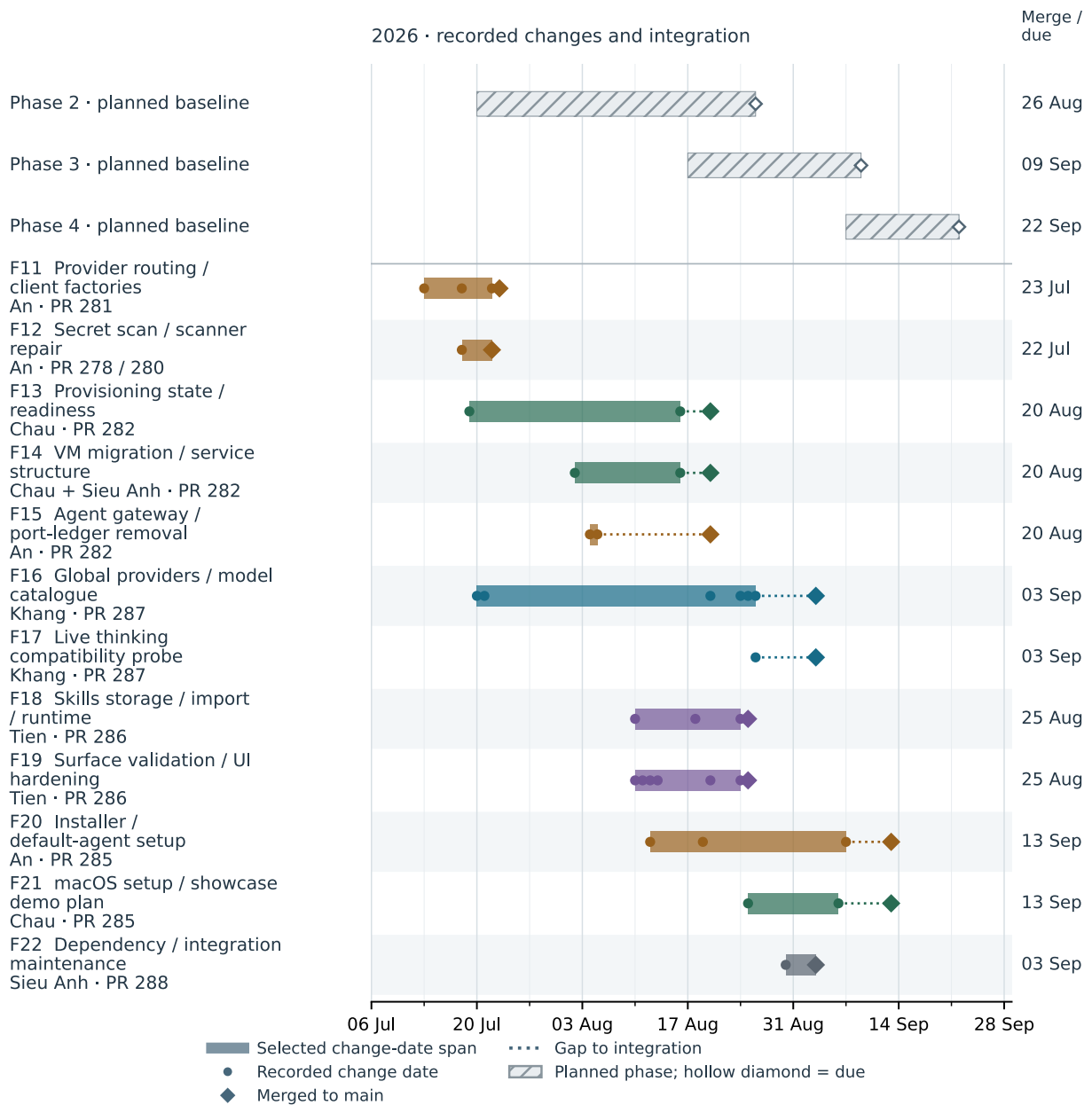


Figure 4: Feature-level Gantt: product completion and hardening. Hatched rows show planned phases; coloured rows show selected change-date spans; filled diamonds mark merges into main. Planned dates use the completion plan's prose and milestone table.

The live probe was introduced in commit 7af467b on 26 August and integrated with PR 287 on 3 September. The commit replaced a curated thinking-capability list with a test request. This is a meaningful refinement beyond the PR's earlier summary, which still describes catalogue-based gating. The final implementation and commit record therefore provide the more precise account of this feature.

4.7.3 Delivery against the completion plan

Table 6 compares planned commitments with observed integration. Phase 2's explicit milestone is 26 August, Phase 3's is 9 September, and Phase 4's is 22 September. The completion plan's embedded Phase 3 Gantt depicts a later endpoint than its prose and milestone table. This report uses the explicit textual dates as its comparison baseline and preserves that discrepancy; it does not infer exact task-level due dates from the conflicting diagram. [3]

Commitment / planned owner	Baseline	Observed record and interpretation
Agent access control / team	Finalise and merge in Phase 2, ending 26 Aug.	PR 277 merged 23 Jul. An's role and delegated-access changes were integrated before the phase milestone.
Skills / team	Finalise and merge in Phase 2.	PR 286 merged 25 Aug. Tien's skill import, binding, runtime and surface validation reached main one day before the milestone.
Provider model fetch / Khang	Phase 2, ending 26 Aug.	PR 287 merged 3 Sep: eight calendar days after the phase milestone. Earlier model discovery was present in An's PR 277; global model management and live probing were later additions.
Knowledge spaces / Chau	Many-to-many shared knowledge spaces in Phase 2.	PRs 265 and 282 establish per-agent knowledge services, followed by VM/gateway work merged 20 Aug. This differs from the planned shared-space topology.
AG-UI parity / Tien; knowledge admin UX / Tien	Further interface work in Phase 2.	PR 264 merged 7 Jul; PR 286 added skills and client-specific rendering on 25 Aug. These records do not establish complete Blazor parity or every planned knowledge-admin improvement.
TTFT / An; performance / Khang; benchmark / Tien	Instrumentation in Phase 2; evaluation in Phase 3.	Retrieval summaries are available with a 14 Sep corpus freeze. No dated run log or equivalent TTFT, cost and human-scoring results establishes full Phase 3 attainment.
VM/runbook / Chau; monitoring/restore / An	Operational hardening in Phase 3.	VM and gateway changes merged 20 Aug; documentation updates are present. No recovery or backup-restore exercise was verified.
Polish, documentation and showcase / team	Phase 4, ending 22 Sep.	Installer and macOS/demo-plan work merged 13 Sep. The 20 Sep report snapshot precedes final showcase acceptance.

Table 6: Planned responsibilities and milestones compared with GitHub integration evidence. A merge verifies integration, not every acceptance criterion.

The record supports a mixed delivery assessment. Access control and skills were integrated before the Phase 2 milestone, while the expanded provider workflow entered main afterwards. The history records continued endpoint, model-selection and thinking-path changes through 26 August, but does not establish why integration occurred on 3 September. A delivery delay can therefore be reported without inventing its cause. Similarly, the 14 September corpus freeze dates an evaluation condition; it is not a benchmark execution date.

4.7.4 Individual contributions and cross-support

Commit-level attribution adds detail to the responsibility areas confirmed by the team. PR authorship alone would understate shared work: PR 265 was opened by An but contains substantial LightRAG work by Chau, while PR 285 includes Chau’s macOS and demonstration-plan changes alongside An’s installer. Table 7 separates primary responsibility from evidenced support. It does not use commit totals as a proxy for effort. [2]

Member	Recorded identity	Primary work and integration	Evidenced supporting work
Vu Gia An	AnVuGia; PepeDaFrog; commit name An Vu.	Role/delegated access, 23 Jul (277); gateway routing within 282, 20 Aug; installer within 285, 13 Sep.	Idle shutdown in 265; early model discovery in 277; provider routing in 281; secret scanning and scanner repair in 278/280.
Nguyen Dinh Minh Chau	Annie-Chau	LightRAG, per-agent containers and document lifecycle in 265, 17 Jul; provisioning and VM migration in 282, 20 Aug.	Legacy-stack cleanup with Tien; macOS installer and showcase demo plan in 285, 13 Sep.
Nguyen Tien	TienNgen	AG-UI, conversational client and generative surfaces in 264, 7 Jul; skills and surface validation in 286, 25 Aug.	Legacy memory/search removal in 265; client capability and access-denial handling; monitoring and architecture documentation in 286.
Tran Nguyen Quy Khang	GarrettTran	Agents-as-tools and inner-agent bindings in 261, 19 Jun; global provider/model management and thinking probes in 287, 3 Sep.	Provider endpoint corrections, model-selection integration and provider discovery error reporting within 287.

Table 7: Primary contribution areas and supporting changes corroborated by selected GitHub commits. Dates identify parent-PR integration into main.

The industrial supervisors also contributed to integration. Nguyen Le Quynh Thien (thiennn) is recorded as the merger of all twelve reviewed PRs. Nguyen Sieu Anh (sieuanhantanbao) contributed LightRAG restructuring within PR 282 and package/integration maintenance in PR 288. These are repository activities; they do not establish meeting attendance or sponsor acceptance of the complete project.

The installer work illustrates collaboration across feature boundaries. PR 285 explicitly depended on the VM branch, and a 7 September integration change adapted default-agent setup to the new provider entity. This links infrastructure, provider configuration and developer onboarding rather than treating installation as an isolated final task. The PR also reports setup checks, but those author-reported checks are not represented here as independently reproduced test results.

NashTech supported LightRAG hosting and access to deployed models. The university provided a US\$300 Claude API allocation for development and testing. These remain team-confirmed resource statements, not expenditure or service-level measurements. Together with the GitHub record, they explain how work was divided and integrated while leaving operational acceptance and remaining evaluation targets explicit.

5 Result analyses and discussions

5.1 Delivered capabilities and engineering findings

The reviewed GitHub history links the final solution to identifiable integration outcomes. Table 8 groups these by the design questions from Section 3, rather than treating repository activity as a single undifferentiated accomplishment. The source snapshot and merge records support implementation and integration claims; the separate retrieval trials support the experimental claims that follow. [1], [2]

Link	Capability	Implemented outcome	Evidence boundary
DQ1	Agent-scoped knowledge	LightRAG/PostgreSQL integration, per-agent containers, asynchronous document status and later VM/gateway routing.	PRs 265/282 and scoped retrieval paths; no independent tenancy-security test.
DQ2	Authorised specialist delegation	Inner-agent bindings and tool invocation with caller-aware exposure and invocation checks.	PRs 261/277 and AgentToolPlugin; composed-answer quality remains to be assessed.
DQ3	Provider and model administration	Global provider configuration, enabled-model selection, endpoint handling and live thinking probes.	PRs 277/281/287; probe acceptance is not a model-quality score.
DQ4	Interactive skills	AG-UI client, declarative surfaces, skill import/binding, template validation and runtime rendering.	PRs 264/286; demonstration packages use synthetic data.
DQ5	Reproducibility and maintenance	Local installer, default-agent setup, provisioning support, scanner configuration and documentation.	PRs 278/280/282/285/288; setup and CI claims need their corresponding execution records.

Table 8: Delivered capabilities mapped to the review’s design questions and their evidence.

Three engineering findings are particularly relevant to the application’s needs. First, capability reuse required access control to be carried into the delegated invocation. Exposing an inner agent as a generic tool was not sufficient: the wrapper also needed the caller’s access decision and the child’s knowledge binding. This joins orchestration and document ownership in a single runtime path.

Second, provider compatibility was resolved at a more specific level than a model-name list. The progression from routing corrections to a live thinking probe shows why provider, endpoint, model and option must be considered together. The implemented result is an administrative check against a real request path. The remaining limitation is that a successful small request does not establish reliable behaviour across all conversations or later provider changes.

Third, reusable UI required more than transporting generated JSON. The skill work added template structure, input-binding validation, stable surface identity and client-aware handling. These changes address the difference between displaying a form and sustaining a usable task flow. The resulting contribution is a stronger interaction contract, with human task evaluation still needed to establish usability and correctness.

5.2 Evaluation evidence and trial conditions

The available quantitative evidence concerns retrieval. The supplied report describes a six-query validation run, a twelve-query pilot and a warm-cache rerun of the same twelve pilot queries. [18] The rerun examines consistency under changed cache conditions; it is not an additional independent query set. Overlap between the six validation questions and the twelve pilot questions is not established.

PR descriptions also report tests, builds and manual checks. Those statements provide development context, but they concern different revisions and procedures and were not independently reproduced during report preparation. In particular, test counts from different PRs are not combined into a final test-suite total. The report retains the distinction between such author-reported checks and the supplied retrieval measurements.

Condition	Supplied trial configuration
Retrieval engine	LightRAG 1.4.16; naive, local, global and hybrid modes.
Generation and embedding	GPT-5.1 through azure_openai; text-embedding-3-large embeddings; no reranker.
Corpus	Four documents, 81 chunks, frozen on 14 September 2026.
Corpus composition	Minimal APIs: 61 chunks; maintainability: 2; medical: 16; handwriting: 2.
Query sets	Validation: 6; pilot: 12; rerun: the same 12 pilot questions.
Cache conditions	Validation and pilot described as cold; rerun used a warm LLM cache.
Available evidence	Summary PDF with charts, aggregate values and query inventory; raw calls, health samples and harness configuration were not supplied.

Table 9: Conditions for interpreting the reported retrieval measurements. Source: supplied retrieval trials, pp. 1–9.

The corpus provides a development-scale mixture of software and non-software topics. Its distribution is uneven, with the Minimal APIs document accounting for 61 of 81 chunks. This makes the trials useful for checking retrieval behaviour across the supplied cases, but does not establish performance on a large or representative enterprise collection. Corpus scale and question composition remain part of the result, rather than background details that can be omitted when reporting hit rates.

The numerical values below are transcribed from the supplied report. They were not recomputed from raw retrieval calls. Captions identify the relevant conditions, and the companion JSON file preserves the plotted values for subsequent verification and editing.

5.3 Retrieval coverage and consistency

All four modes achieved a reported hit@5 of 1.000 on the six-query validation set and 0.917 on the twelve-query pilot. The pilot value corresponds to eleven successful cases out of twelve and exceeds the completion plan’s numerical hit@5 threshold of 0.80 on this set. Figure 5 shows both hit@1 and hit@5, preserving the difference between placing relevant material first and including it within a larger candidate set. [18]

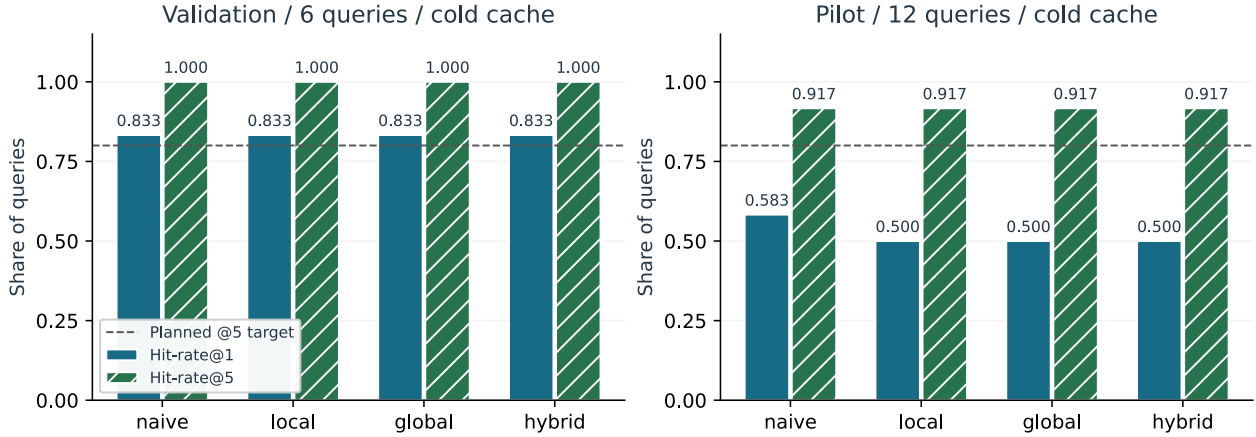


Figure 5: Reported retrieval coverage by mode and query set. The dashed line is the planned hit@5 target, not a hit@1 requirement. Pilot values were unchanged in the warm-cache rerun. Source: retrieval trials, pp. 2–7.

The pilot’s hit@1 was 0.583 for naive retrieval and 0.500 for local, global and hybrid retrieval, corresponding to seven and six successful first-ranked cases respectively. On this task set, the aggregate results therefore do not demonstrate an advantage for hybrid retrieval. At hit@5, hybrid matched the best observed mode; at hit@1, naive had a one-query advantage. The small absolute difference should be interpreted at the query level rather than presented as a general ranking of retrieval approaches.

Consistency across the pilot and warm-cache rerun is a useful result. The reported hit rates were preserved despite the changed cache condition, suggesting stable aggregate retrieval coverage for those repeated questions. It does not establish stability across new questions, corpus updates or repeated independent deployments.

The source uses both document-level and chunk-level language when describing top-k hits. With only four documents, a top-five list of unique documents cannot be interpreted literally in the same way as five retrieved chunks. Until the raw records and scorer are available, the report retains the source’s term “reported hit@5.” A hit associated with the expected document should not automatically be read as retrieval of the exact answer-bearing passage.

The most informative unsuccessful case is the captive-dependency question, identified as p04 in the query inventory. It was missed by all four modes. The shared failure makes the case useful for inspecting the source passage, chunk construction and expected-answer mapping before attributing the problem to a particular retrieval mode. Its cause cannot be established from the aggregate report alone.

5.4 Latency tradeoffs and cache conditions

The trials expose a clear latency tradeoff even where coverage is equal. In the cold pilot, relative median retrieval latency was 1.0 for naive, 3.4 for local, 3.3 for global and 3.9 for hybrid. In the warm-cache rerun, the corresponding ratios were 1.0, 1.3, 1.3 and 1.8. Figure 6 normalises each trial to its own naive baseline. [18]

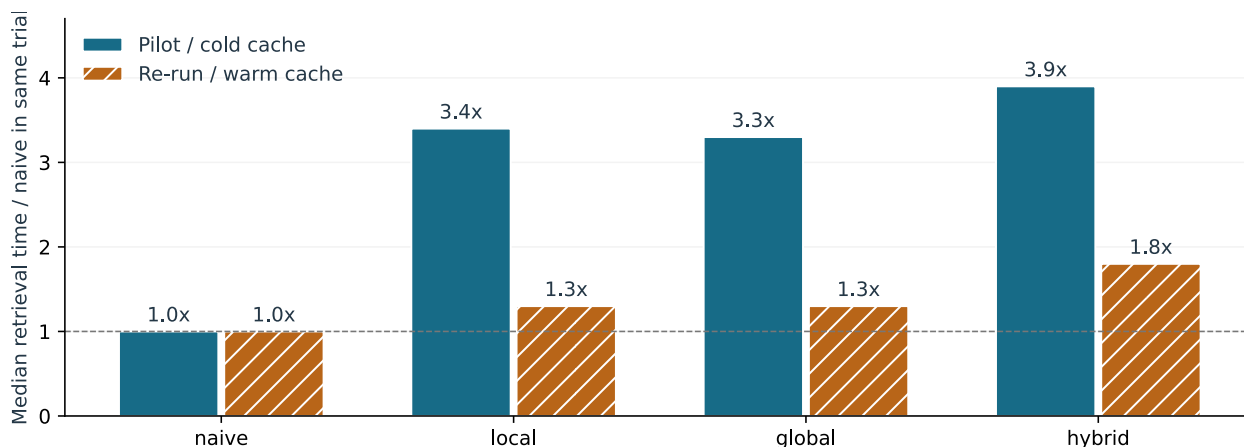


Figure 6: Reported relative median retrieval latency. Each trial has its own naive = 1 baseline; bar differences across trials are not absolute time savings. Source: retrieval trials, pp. 4–7.

These results support testing naive retrieval as a lower-latency baseline for similar queries, since the graph-based modes did not improve aggregate hit@5 in this pilot. They do not justify removing graph-based retrieval from the platform. Questions requiring broader relationship synthesis may behave differently, and the trial does not isolate the effect of every processing stage.

The source also describes answer generation as approximately seven times faster in the warm-cache rerun. This is useful evidence that cache state materially affects the observed experience, but the supplied summary is insufficient to separate model, retrieval, network and cache contributions. The relative retrieval ratios should not be converted into dollar costs, nor should a warm-cache answer-time observation be equated with a time-to-first-token percentile.

Health checks succeeded in 9 of 9 validation samples, 44 of 44 pilot samples and 32 of 32 rerun samples. This supports the narrower observation that the sampled checks were successful during the trials. The sample intervals and observation-window durations are not supplied, so these counts do not establish availability across the full demonstration window or compliance with the recovery target.

The operational implication is to make cache state and measurement boundaries visible in subsequent comparisons. A repeatable evaluation should report absolute timings, distinguish retrieval from full response generation, and record both cold and warm conditions. This would allow the team to connect the observed tradeoffs to the completion plan’s service-level targets without changing those targets after the fact.

5.5 Comparison with planned success criteria

Table 10 separates observed results from requirements that need further evidence. This preserves the positive retrieval finding while avoiding an unsupported claim that all performance goals were achieved. [3], [18]

Criterion	Planned target	Evidence and current assessment
Retrieval coverage	Hybrid hit@5 ≥ 0.80 and at least the best single mode.	Reported pilot hit@5 = 0.917 for all modes. Numerical target met on this set; scoring unit requires confirmation.
Answer quality	Mean human rating $\geq 4/5$; two independent blind raters.	No validated human-rated output results supplied. Retrieval hits cannot substitute for this measure.
Ingestion	P95 ≤ 20 s below 100 KB; ≤ 90 s for 100 KB–1 MB; ≤ 5 min above 1 MB.	No comparable ingestion measurements supplied for these document-size bands.
Response latency	TTFT p50 ≤ 1.5 s, p95 ≤ 4 s; full response p50 ≤ 12 s.	Relative retrieval medians and a cache observation do not establish these end-to-end targets.
Availability and recovery	Full demonstration-window availability; recovery ≤ 60 s.	Successful sampled checks reported; no complete window or recovery exercise supplied.
Cost	Median $\leq$ US\$0.03 per five-turn conversation.	No comparable five-turn cost distribution supplied. Latency ratios are not cost measurements.
Quality gate	Green quality checks for each merge.	CI workflow present in source; execution history and merge-by-merge compliance not verified.

Table 10: Success criteria retained from the completion plan and assessed against the available evidence.

5.6 Priorities for further validation

The first priority is to preserve and inspect the trial artefacts: query definitions, relevance labels, raw calls, configuration and health samples. This would resolve the hit-definition ambiguity and permit recomputation of the published aggregates. The p04 failure should then be analysed against the exact source passage and retrieved chunks. A corrected scoring definition may change interpretation without changing the underlying system.

The second priority is a small, explicitly judged answer set covering supported answers, missing information and questions that require specialist delegation. Human assessment should examine correctness, use of evidence and appropriate refusal or uncertainty. Retrieval modes should be compared under matched conditions; broader conclusions require a more representative corpus and question distribution.

The third priority is operational assurance: role-based delegation tests, absolute latency traces, ingestion trials, recovery exercises and conversation-level cost records. Each activity corresponds to an existing requirement. This keeps future work focused on demonstrating the platform’s intended behaviour rather than expanding the feature list before the present claims can be validated.

6 Summary and reflection

NTG-agent-vnext brings scoped knowledge retrieval, specialist delegation, configurable model access and interactive agent responses into a common platform. The inspected implementation provides evidence for these integrations. The retrieval trials add a concrete outcome: strong reported top-five coverage on the supplied pilot and unchanged hit rates in a warm-cache rerun, together with measurable differences in relative retrieval latency. The evidence supports these bounded findings while leaving human answer quality and several operational targets for subsequent validation.

From the team's perspective, the most consequential design lesson is that access boundaries must shape orchestration. Moving toward a dedicated knowledge context for each agent clarified what a specialist owns and how another agent may use its capability. The resulting delegation path has an understandable contract: authorise the call, let the inner agent retrieve within its own context, and return an answer. The remaining lesson is to test this contract explicitly, including denied and revoked access, rather than relying only on a successful demonstration.

Provider integration produced a related lesson about configuration. A model name or provider label does not fully describe which request options an endpoint accepts. The thinking probe turns part of that uncertainty into a concrete compatibility check. Its interpretation must remain precise: a successful probe validates the tested request, while quality and reliability require separate observations. Future work should preserve the tested configuration and failure reason so administrators can distinguish unsupported options from temporary connection problems.

Local environment setup was another practical episode. The team reported friction from collecting configuration values and preparing services, and introduced an interactive setup script with defaults for non-sensitive internal settings. The lesson is to treat developer onboarding as part of the deliverable. A documented, repeatable setup path helps future contributors understand the system, even though its time savings have not yet been measured. Independently repeating the fresh-machine checks reported in PR 285 would provide a direct next validation.

The skill work added a lesson about interactive correctness. PR 286 records changes to seeded input bindings, stable surface identifiers and client-capability handling. From the team's perspective, these issues show that a convincing generated answer and a usable workflow are different deliverables. Future acceptance criteria should include a complete user journey across several turns, including input edits and continuation, alongside checks of the generated content. [2]

The evaluation experience shows why measurement preparation should begin alongside implementation. The supplied trial summaries are already useful for analysing coverage and latency tradeoffs, but absent raw records and unresolved metric wording limit what can be independently reconstructed. Human assessment was not completed within the available time. In a future iteration, the team would define the scoring unit, retain executable evaluation configurations and reserve time for judging outputs before the final reporting stage.

The final priorities are therefore consolidation and validation. The team should confirm outstanding delivery and contribution records, preserve the retrieval artefacts, evaluate answers and access boundaries, and test the remaining operational criteria under documented conditions. This would build on the implemented platform and the initial retrieval findings while producing stronger evidence for deployment and continued development.

References

- [1] NTG Agent Project, “NTG Agent: Source Snapshot and Implementation Documentation.” [Online]. Available: <https://github.com/nashtech-garage/ntg-agent>
- [2] NTG Agent Contributors, “NTG Agent Pull Requests and Commit History: PRs 261, 264, 265, 277, 278, 280, 281, 282, 285, 286, 287 and 288.” [Online]. Available: <https://github.com/nashtech-garage/ntg-agent/pulls?q=is%3Apr+is%3Amerged>
- [3] QUOTA Team, “Capstone Project Completion Plan Report.” 2026.
- [4] P. Lewis, E. Perez, A. Piktus, and others, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, 2020, [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [5] N. F. Liu *et al.*, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.
- [6] D. Edge, H. Trinh, N. Cheng, and others, “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” *arXiv preprint arXiv:2404.16130*, 2024, [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [7] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and Fast Retrieval-Augmented Generation,” *arXiv preprint arXiv:2410.05779*, 2024, [Online]. Available: <https://arxiv.org/abs/2410.05779>
- [8] D. F. Ferraiolo and D. R. Kuhn, “Role-Based Access Controls,” in *15th National Computer Security Conference*, 1992. [Online]. Available: <https://www.nist.gov/publications/role-based-access-controls>
- [9] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [10] LangGenius, “Dify: Open-Source LLM App Development Platform.” [Online]. Available: <https://github.com/langgenius/dify>
- [11] Microsoft, “Microsoft Agent Framework Overview.” [Online]. Available: <https://learn.microsoft.com/en-us/agent-framework/overview/>
- [12] LangChain, “LangGraph Overview.” [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [13] AG-UI, “Introduction to the Agent–User Interaction Protocol.” [Online]. Available: <https://docs.ag-ui.com/introduction>
- [14] A2UI, “Agent-to-User Interface.” [Online]. Available: <https://a2ui.org/>
- [15] Model Context Protocol, “Architecture Overview.” [Online]. Available: <https://modelcontextprotocol.io/docs/2026-07-28/learn/architecture>
- [16] Agent Skills, “Agent Skills: Overview.” [Online]. Available: <https://agentskills.io/home>
- [17] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAs: Automated Evaluation of Retrieval Augmented Generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158. doi: 10.18653/v1/2024.eacl-demo.16.
- [18] NTG Agent Project, “NTG Agent Retrieval Trials.” 2026.
- [19] T. Kassis, V. Agarwal, Y. He, D. Patel, and A. M. Brueckner, “Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents,” *arXiv preprint arXiv:2609.00065*, 2026, [Online]. Available: <https://arxiv.org/abs/2609.00065>

Appendices

Appendix A. Implementation evidence map

Paths below are relative to `ntg-agent/`. The abbreviation `Orch/` denotes `NTG.Agent.Orchestrator/`. They identify the basis for implementation claims, not proof of a passing runtime test or individual authorship. Team responsibility areas and resource support are user-confirmed statements recorded in the report workspace's `process.md`. GitHub commit attribution and PR integration dates are now recorded in Appendix D; exact task-start dates, effort and deployment acceptance are not inferred from that history.

Claim or workflow	Source location
Access decisions	<code>Orch/Services/Agents/AgentAccessService.cs</code>
Agent construction and tool exposure	<code>Orch/Services/Agents/AgentFactory.cs</code>
Authorised child invocation and scoped retrieval	<code>Orch/Plugins/AgentToolPlugin.cs</code>
Knowledge routing and readiness	<code>NTG.Agent.LightRag/LightRagClientFactory.cs</code>
Container and workspace configuration	<code>NTG.Agent.LightRag/LightRagContainerManager.cs</code>
Document ingestion	<code>NTG.Agent.LightRag/LightRagKnowledge.cs</code>
Thinking compatibility probe	<code>Orch/Services/Agents/ThinkingSupportProbe.cs</code>
Provider model discovery	<code>Orch/Services/Agents/ProviderModelService.cs</code> <code>Orch/Controllers/ProviderProbeController.cs</code>
Interactive interface and skills	<code>docs/A2UI-and-AG-UI.md</code> <code>Orch/Services/Skills/</code> (<code>SkillPrompt</code> , <code>SkillPackageImporter</code> , <code>SurfaceValidator</code>)
Telemetry and health configuration	<code>NTG.Agent.ServiceDefaults/Extensions.cs</code> <code>docs/monitoring-setup.md</code>
Local setup and CI configuration	<code>install-local.sh</code> <code>Orch/Services/Agents/DefaultAgentProviderSeeder.cs</code> <code>.github/workflows/ntg-agent-ci.yml</code>

Table 11: Source map for the implementation discussion.

Documentation is used to explain intended workflows; selected code paths substantiate their implementation. The report preparation did not execute the application, run access-control tests, validate a deployed service or reproduce the benchmark. The report's compilation checks concern document integrity and do not change these implementation-evidence limits.

Appendix B. Retrieval data and reproducibility

The figure data are transcribed into `report/data/retrieval-summary.json`. The accompanying generation script recreates the plots without estimating values from unlabelled bars. Table 12 retains the aggregate values used in the discussion. Mode order is naive, local, global and hybrid throughout.

Trial	Queries	Hit@1	Hit@5	Median latency ratios
Validation (cold)	6	0.833 for all	1.000 for all	1.0 / 3.7 / 3.5 / 4.0
Pilot (cold)	12	0.583 / 0.500 / 0.500 / 0.500	0.917 for all	1.0 / 3.4 / 3.3 / 3.9
Rerun (warm LLM cache)	Same 12	0.583 / 0.500 / 0.500 / 0.500	0.917 for all	1.0 / 1.3 / 1.3 / 1.8

Table 12: Transcribed aggregate retrieval results. Each latency row is normalised to its own naive baseline. Source: retrieval trials, pp. 2–7.

The raw call logs, health samples, relevance labels, checksums and benchmark harness referenced by the supplied summary are not included in the current evidence package. Their absence prevents independent recomputation of the aggregates and attribution of latency changes to particular stages. The supplied PDF also uses both document and chunk terminology for hits. A reproducible extension should define the scored unit, preserve retrieved identifiers and ranks, and identify the answer-bearing evidence for each question.

The rerun reuses the pilot questions and should not be counted as twelve new questions. No uncertainty interval or statistical significance claim is added because the summary is not sufficient to establish the sampling design. The withdrawn non-human answer scores are not used as evidence of correctness. A future human assessment should retain its rubric, judge instructions and per-answer decisions.

Appendix C. Report production and final review

The report is authored in Typst. Architecture, delegation, schedule and retrieval figures are generated as vector graphics with Matplotlib from the source map, planned dates and transcribed trial values. The Gantt figures distinguish planned phases, recorded feature-change dates and PR integration dates. Figure-generation guidance was informed by the Scientific Agent Skills library. [19] This tooling acknowledgement concerns report preparation, not a dependency of NTG-agent-vnext.

The source package includes independently editable weekly journal files. Proposed dates, attendance variations and actions must be confirmed before those drafts can serve as final meeting records. Other final-review items are exact submission metadata, the actual completion status at the final milestone, any further contribution or deployment evidence, and any additional evaluation artefacts supplied after the 20 September snapshot.

Appendix D. GitHub history and timeline method

The register below identifies the twelve PRs inspected on 20 September 2026. Dates use UTC+7 and the format month–day within 2026. Each PR number links to the original record. Merge SHAs match the local application’s integration history. [2]

PR	Author	Opened	Merged	Merge SHA
PR 261	GarrettTran	06-01	06-19	5ec09cd
PR 264	TienNgen	06-12	07-07	ec4b312
PR 265	AnVuGia	06-29	07-17	21c4684
PR 277	AnVuGia	07-17	07-23	0443991
PR 278	AnVuGia	07-18	07-22	21030d7
PR 280	AnVuGia	07-22	07-22	1792969
PR 281	AnVuGia	07-22	07-23	b8b2304
PR 282	Annie-Chau	08-02	08-20	4823553
PR 285	AnVuGia	08-19	09-13	7789ed0
PR 286	TienNgen	08-20	08-25	954f497
PR 287	GarrettTran	08-20	09-03	9847237
PR 288	sieunhantanbao	08-30	09-03	33733bb

Table 13: GitHub PR register for the feature timeline and contribution analysis.

The source data are preserved in `report/data/github-history.json`, with a compact `github-pr-register.csv` alongside them. The feature mapping in `delivery-timeline.json` identifies the selected commit SHAs for each of F01–F22. The generator retains commit-author dates as recorded and separately stores committer timestamps, because rebasing and cherry-picking can change the latter. The ranges describe the selected evidence, not the duration of all work on a feature.

PR commit lists can include inherited changes. For example, the `role-access` and `installer` branches contain earlier LightRAG work. The mapping selects changes relevant to each row rather than assigning every ancestor to the later feature. Multiple rows can share a PR integration date; their ranges must not be added together as person-days. Likewise, the duplicated `macOS/demo-plan` changes in PR 285 are not interpreted as two independent deliverables.

The local application’s reviewed head is 7789ed0, dated 13 September. PR bodies are useful decision records but may become stale: PR 287’s description retains catalogue-based thinking gating, whereas its 26 August commit 7af467b removes that catalogue and adds the live probe. The final code and specific commit take precedence for that implementation claim.

The completion plan’s Phase 3 graphic and explicit textual milestone disagree. The Gantt uses the prose/table interval of 17 August–9 September and flags the conflict in Section 4.7. Task names and planned assignees are taken from the plan, but precise subtask dates are not fabricated from the inconsistent graphic. Benchmark execution dates, deployment acceptance and work without a GitHub record remain separate evidence requirements.

Meeting journals

The twelve weekly journal files are included below for team review. Their proposed scheduling and draft actions require confirmation; they are not used to establish measured performance, individual delivery dates, or sponsor acceptance. Each week remains independently editable in the accompanying source package. The final scheduled week falls after this report's evidence snapshot.

RMIT UNIVERSITY / QUOTA TEAM

Baseline Review and Responsibility Alignment

Capstone Project B - Meeting No. 1 / Week 1
NTG-agent-vnext | Baseline consolidation before Phase 2

Meeting Details

Date:	06/07/2026 (proposed weekly slot)
Time:	10:00 AM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review the existing NTG Agent platform and carry forward the revised scope from earlier work. Separate inherited capabilities from this semester's contributions.
2	Use the reported workstreams: Chau and An on LightRAG, isolation and permissions; Tien on AG-UI and skills; Khang on agents-as-tools and provider integration.
3	Prepare a weekly feature checklist. Exact completion dates and supporting contributions require team confirmation.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
1.1	Map current features to the completion-plan scope.	All members	13/07/2026
1.2	Identify LightRAG and access-control work still required.	Chau, An	13/07/2026
1.3	Outline integration dependencies across agent tools and UI.	Khang, Tien	13/07/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Knowledge Isolation and Specialist-Agent Review

Capstone Project B - Meeting No. 2 / Week 2
NTG-agent-vnext | Baseline consolidation before Phase 2

Meeting Details

Date:	13/07/2026 (proposed weekly slot)
Time:	2:00 PM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review agent-level knowledge isolation using separate LightRAG instances instead of relying on document tags. The user confirms this architectural change; its discussion date remains unverified.
2	Review the user-confirmed flow: with appropriate client permissions, the outer agent calls an inner agent as a tool. The inner agent queries its own KB and returns an answer.
3	Stakeholder question for validation: how should the team balance stronger knowledge separation against instance management and infrastructure overhead? No mentor response is recorded.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
2.1	Document the isolation boundary and instance lifecycle.	Chau, An	20/07/2026
2.2	Summarise the specialist invocation flow and its constraints.	Khang	20/07/2026
2.3	List UI events needed to expose agent activity.	Tien	20/07/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Phase 2 Planning and Integration Boundaries

Capstone Project B - Meeting No. 3 / Week 3
NTG-agent-vnext | Phase 2: product completeness

Meeting Details

Date:	20/07/2026 (proposed weekly slot)
Time:	10:00 AM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review the Phase 2 target of 26 August and identify outstanding skills, access-control, provider-discovery and client-integration work.
2	Keep each agent's KB isolated while enabling authorised outer-to-inner delegation. This flow does not itself implement the completion plan's many-to-many agent/container mapping.
3	Use role-based workstreams to coordinate integration, without assuming that a planned milestone proves release readiness.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
3.1	Prepare the Phase 2 feature checklist with evidence links.	All members	27/07/2026
3.2	Document delegated KB access separately from storage sharing.	Chau, An	27/07/2026
3.3	Align skill and provider settings with agent configuration.	Tien, Khang	27/07/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Document Lifecycle and Access Control

Capstone Project B - Meeting No. 4 / Week 4
 NTG-agent-vnext | Phase 2: knowledge and permissions

Meeting Details

Date:	27/07/2026 (proposed weekly slot)
Time:	10:00 AM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review how document upload, ingestion status and removal relate to the correct agent knowledge base.
2	Review permission boundaries for administrative actions and user access. Keep access control distinct from physical knowledge-store separation.
3	Prepare functional examples for authorised access and attempted cross-agent access. These are proposed checks, not recorded passing results.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
4.1	Review document lifecycle behaviour and record open issues.	Chau, An	03/08/2026
4.2	Trace access checks to the relevant application paths.	Chau, An	03/08/2026
4.3	Coordinate permission behaviour with the client workflow.	Tien, Khang	03/08/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

AG-UI Integration and Administrator-Defined Skills

Capstone Project B - Meeting No. 5 / Week 5
 NTG-agent-vnext | Phase 2: client and skill workflow

Meeting Details

Date:	03/08/2026 (proposed weekly slot)
Time:	2:00 PM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review AG-UI event streaming and the presentation of structured agent output. Explain transport and rendered UI responsibilities separately.
2	Review administrator-defined SKILL.md packages, including how skills are selected for an agent and made available during a request.
3	Stakeholder question for validation: are the authoring and assignment steps clear enough for an administrator to repeat? No sponsor usability judgement is asserted.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
5.1	Document the skill authoring and assignment workflow.	Tien	10/08/2026
5.2	Check skill integration with agent tool configuration.	Tien, Khang	10/08/2026
5.3	Review access boundaries for skill administration.	Chau, An	10/08/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Provider Discovery and Thinking-Mode Probes

Capstone Project B - Meeting No. 6 / Week 6
NTG-agent-vnext | Phase 2: model configuration

Meeting Details

Date:	10/08/2026 (proposed weekly slot)
Time:	10:00 AM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review available-model fetching for configured providers so administrators can select models supported by their endpoint.
2	Review the reported thinking-mode probe: send a test request using the same thinking parameter as the real payload before enabling the setting.
3	Distinguish endpoint compatibility from answer quality. A successful probe does not establish that one model reasons better than another.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
6.1	Document model discovery and probe request construction.	Khang	17/08/2026
6.2	Check settings feedback for unsupported configurations.	Khang, Tien	17/08/2026
6.3	Review access to provider settings and credentials.	Chau, An	17/08/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Feature Integration and Operational Preparation

Capstone Project B - Meeting No. 7 / Week 7
 NTG-agent-vnext | Phase 2 / Phase 3 overlap

Meeting Details

Date:	17/08/2026 (proposed weekly slot)
Time:	2:00 PM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review integration across knowledge isolation, specialist invocation, provider settings and skills before the feature-complete target.
2	Review operational needs for the NashTech-supported LightRAG infrastructure and model deployments. Exact deployment identifiers and support dates still need confirmation.
3	Prepare an evidence checklist for monitoring, health checks and handover. Operational readiness requires records beyond the existence of configuration files.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
7.1	Review the LightRAG deployment and health-check notes.	Chau, An	24/08/2026
7.2	Check integration issues across providers, skills and UI.	Khang, Tien	24/08/2026
7.3	Collect reproducible setup and handover requirements.	All members	24/08/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Feature-Complete Milestone Review

Capstone Project B - Meeting No. 8 / Week 8
 NTG-agent-vnext | Phase 2 milestone: 26 August (planned)

Meeting Details

Date:	24/08/2026 (proposed weekly slot)
Time:	2:00 PM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	None proposed; verify with team
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review the planned 26 August milestone against the actual feature checklist. Keep unfinished items visible rather than marking the milestone complete by date alone.
2	Check skill, access-control and provider-discovery integration. Verify delegated KB access, the planned storage mapping and TTFT instrumentation against code evidence.
3	Reserve benchmark and quality claims for recorded measurements. This draft does not establish a release, passing test run or sponsor acceptance.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
8.1	Record implemented, incomplete and deferred deliverables.	All members	31/08/2026
8.2	Confirm the final knowledge mapping and permission scope.	Chau, An	31/08/2026
8.3	Collect provider and skill demonstration evidence.	Khang, Tien	31/08/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Local Setup Automation and Developer Onboarding

Capstone Project B - Meeting No. 9 / Week 9
NTG-agent-vnext | Phase 3: operational preparation

Meeting Details

Date:	31/08/2026 (proposed weekly slot)
Time:	10:00 AM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	An (proposed absence; verify with team)
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review the user-reported friction in local installation and configuration. The team introduced an automation script to simplify setup for future developers.
2	Review prompts for required API and environment inputs, with defaults for non-sensitive internal settings. Keep credentials out of source and report examples.
3	Record the expected onboarding benefit without inventing a measured time saving. Script ownership, delivery date and any fresh-machine checks require confirmation.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
9.1	Review setup prompts, defaults and documentation.	All members	07/09/2026
9.2	Identify the script author and supporting contributors.	All members	07/09/2026
9.3	List prerequisites and unresolved onboarding issues.	All members	07/09/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Evaluation Gap and Delivery Priorities

Capstone Project B - Meeting No. 10 / Week 10

NTG-agent-vnext | Phase 3 / Phase 4 overlap

Meeting Details

Date:	07/09/2026 (proposed weekly slot)
Time:	10:00 AM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	Chau (proposed absence; verify with team)
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review the planned 9 September production-readiness milestone and identify missing evidence. Its achievement has not been confirmed.
2	The user later confirmed on 20 September that formal benchmarking and AI-output evaluation remained incomplete due to limited time to research the evaluation approach. Validate when this issue first arose.
3	Proposed follow-up: prioritise accurate implementation evidence and document the evaluation gap. Do not replace measured findings with estimated scores or general demo impressions.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
10.1	Record unresolved evaluation and operational evidence gaps.	All members	14/09/2026
10.2	Collect available functional examples without quality claims.	Tien, Khang	14/09/2026
10.3	Update infrastructure and handover limitations.	Chau, An	14/09/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Report Evidence, Resources and Handover Review

Capstone Project B - Meeting No. 11 / Week 11
 NTG-agent-vnext | Phase 4: documentation and polish

Meeting Details

Date:	14/09/2026 (proposed weekly slot)
Time:	2:00 PM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, Chau. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; attendance confirmed by user).
Apologies:	An (proposed absence; verify with team)
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Review feature evidence, member contributions and remaining handover notes. The report should explain both delivered capabilities and outstanding evaluation.
2	The user later reported NashTech infrastructure/model support and US\$300 of university Claude API support. Verify exact names and budget wording before final attribution.
3	Use the setup-automation experience as a concrete reflection example. Any additional sponsor feedback or teamwork lessons must be confirmed rather than invented.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
11.1	Review contribution mapping and reconstructed weekly records.	All members	21/09/2026
11.2	Verify infrastructure and model-support descriptions.	Chau, An, Khang	21/09/2026
11.3	Collect UI/skill evidence and refine the demonstration flow.	Tien	21/09/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.

RMIT UNIVERSITY / QUOTA TEAM

Final Report and Showcase Readiness

Capstone Project B - Meeting No. 12 / Week 12
NTG-agent-vnext | Phase 4: proposed pre-Showcase agenda

Meeting Details

Date:	21/09/2026 (proposed)
Time:	2:00 PM (UTC+7; proposed slot)
Attendees:	QUOTA (proposed): Khang, Tien, An. Nguyen Le Quynh Thien and Nguyen Sieu Anh (Industrial Supervisors; expected).
Apologies:	Chau (proposed absence; verify with team)
Copy To:	Proposed: team repository / shared drive; distribution unconfirmed.

Information / Decision

Item No.	Discussion Summary (draft wording)
1	Proposed agenda: review the report against OENG1185 requirements, including fewer than 20 assessed pages and the stated exclusions.
2	Proposed agenda: confirm implemented deliverables, unresolved limitations and the incomplete benchmark status. Do not state that all success criteria were met.
3	Proposed agenda: validate the retrospective journals, rehearse the demonstration and confirm handover arrangements for the planned 22 September Showcase. No meeting outcome is recorded.

Action Items

Suggested follow-ups, owners and due dates; completion is not recorded.

No.	Item	Who	By
12.1	Confirm factual claims, page count and final PDF layout.	All members	22/09/2026
12.2	Correct journals and mark confirmed retrospective entries.	All members	22/09/2026
12.3	Confirm demonstration ownership and handover materials.	All members	22/09/2026

Validation: Time and QUOTA attendance are draft allocations for review. Supervisor attendance is user-confirmed for past meetings; future attendance is expected.